

將 Spring Boot Kotlin 應用程式容器化，並部署至 Cloud Run

來源：<https://codelabs.developers.google.com/codelabs/cloud-kotlin-jib-cloud-run?hl=zh-tw>

步驟數：8

瞭解如何在不執行 Docker 或 Dockerfile 的情況下，立即建構並發布適用於 Spring Boot Kotlin 應用程式的最佳化 Docker 映像檔，然後在 Cloud Run 執行建構的映像檔。

目錄

1. [1. 事前準備](#)
2. [2. 開始設定](#)
3. [3. 初始化 Spring Boot 應用程式](#)
4. [4. 新增網頁控制器](#)
5. [5. 將應用程式容器化並發布至 Container Registry](#)
6. [6. 在 Cloud Run 執行容器化應用程式](#)
7. [7. 清除所用資源](#)
8. [8. 恭喜](#)

步驟 1 · 約 1 分鐘

1. 事前準備

Google 提供[功能強大的映像檔建構工具](#)，您可輕鬆建構及發布 Java 應用程式的 Docker 容器最佳化映像檔，無需使用 Docker 或 Dockerfile，Google Cloud 也透過 [Cloud Run](#) 將無伺服器概念導入容器。Cloud Run 是一個代管運算平台，可自動調整無狀態容器的資源配置。在本程式碼研究室中，您會發現將 Spring Boot Kotlin 應用程式容器化、發布至 Container Registry，以及在 Google Cloud 上順暢執行映像檔有多麼容易！

這個程式碼研究室會逐步引導您在 Kotlin 中設定簡單的應用程式，並示範如何使用 Google Cloud 服務和工具，包括 [Jib](#)、[Container Registry](#) 和 [Cloud Run](#)。

必要條件

- 熟悉 Java 程式設計語言和工具
- 瞭解標準 Linux 文字編輯器，例如 Vim、Emacs 和 nano

學習內容

- 設定 Spring Boot Kotlin 應用程式。
- 建構經過最佳化的 Docker 映像檔。
- 將映像檔發布至 Container Registry。
- 在 Cloud Run 上執行容器化應用程式。

軟硬體需求

- 具備 Google Cloud 專案
 - 瀏覽器，例如 Google Chrome
-

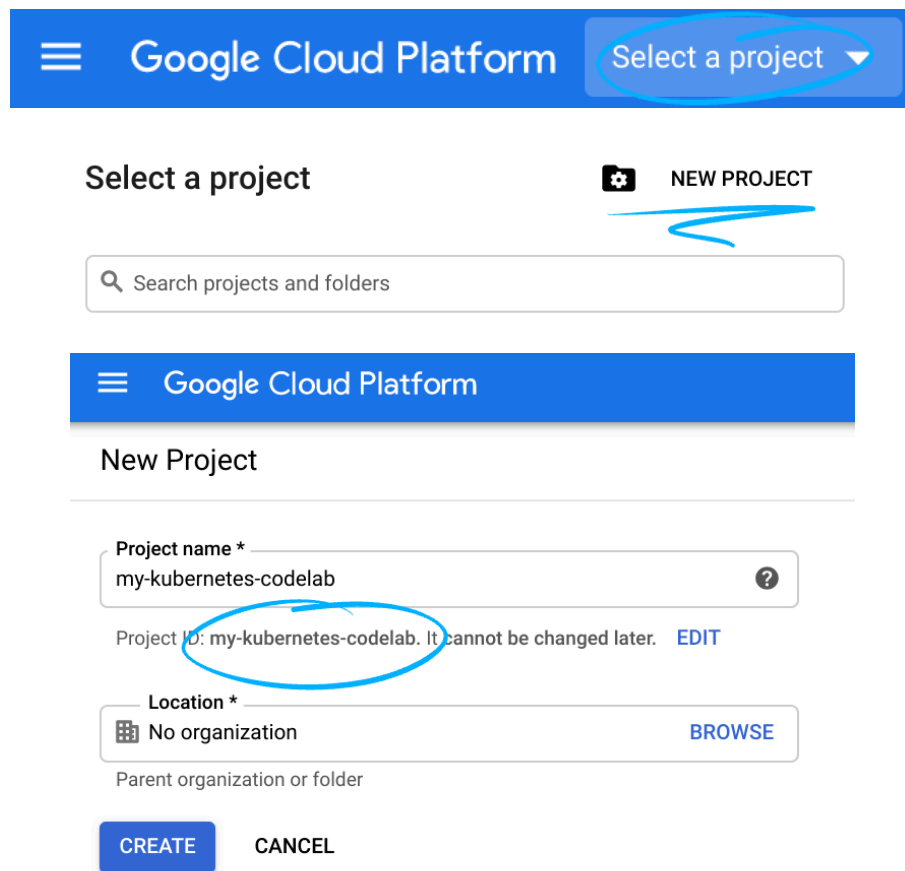
步驟 2 · 約 2 分鐘

2. 開始設定

自修實驗室環境設定

1. 登入 [Cloud 控制台](#)，建立新專案或重複使用現有專案。如果沒有 Gmail 或 Google Workspace 帳戶，請先[建立帳戶](#)。

注意：記住以下網址，即可輕鬆存取 Cloud 控制台：`console.cloud.google.com`。



The screenshot shows the Google Cloud Platform interface for creating a new project. At the top, there's a blue header with the Google Cloud Platform logo and a 'Select a project' dropdown. Below this, there's a 'Select a project' section with a search bar and a 'NEW PROJECT' button. The 'New Project' form is displayed below, with fields for 'Project name *' (filled with 'my-kubernetes-codelab'), 'Project ID' (filled with 'my-kubernetes-codelab'), and 'Location *' (filled with 'No organization'). There are 'CREATE' and 'CANCEL' buttons at the bottom.

請記住專案 ID，這是所有 Google Cloud 專案中不重複的名稱 (上述名稱已遭占用，因此不適用於您，抱歉！)。本程式碼研究室稍後會將其稱為 `PROJECT_ID`。

注意：如果您使用 Gmail 帳戶，可以將預設位置設為「沒有機構」。如果您使用 Google Workspace 帳戶，請選擇適合貴機構的位置。

2. 接著，您必須在 Cloud 控制台中[啟用帳單](#)，才能使用 Google Cloud 資源。

完成本程式碼研究室的費用應該不高，甚至完全免費。請務必按照「清除」部分的指示操作，瞭解如何停用資源，避免在本教學課程結束後繼續產生帳單費用。Google Cloud 新使用者可參加[價值\\$300 美元的免費試用計畫](#)。

Cloud Shell

雖然您可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Cloud Shell](#)，這是 Google Cloud 中執行的指令列環境。

啟用 Cloud Shell

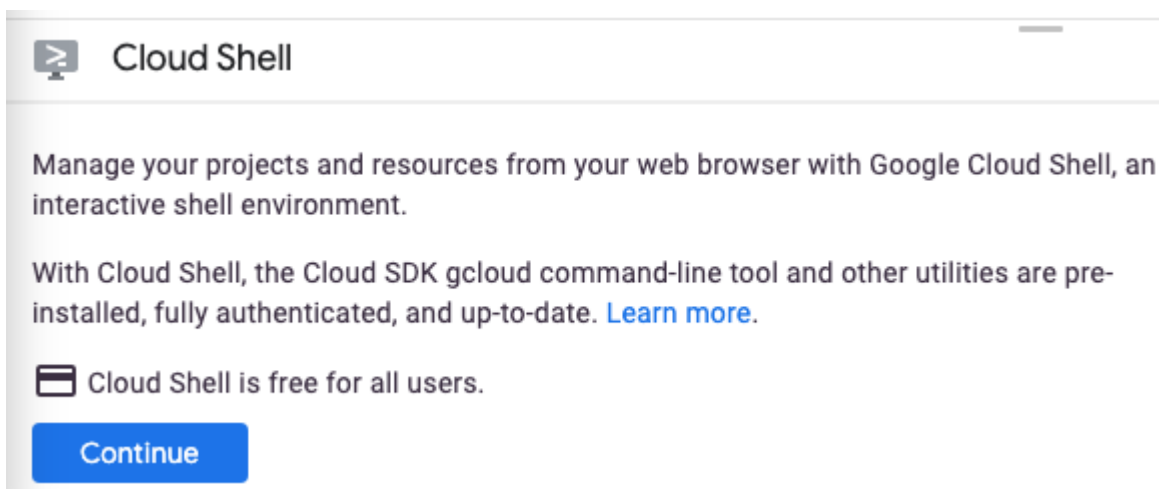
1. 在 Cloud 控制台，點選「啟用 Cloud Shell」圖示



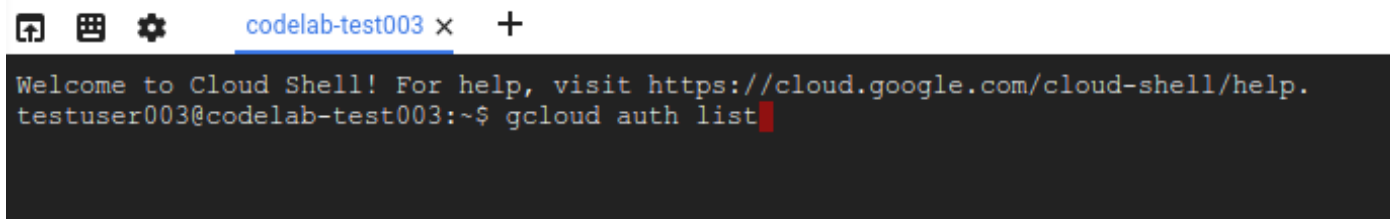
。



如果您是首次啟動 Cloud Shell，系統會顯示中繼畫面 (位於摺疊式選單下方)，說明這個指令列環境。點選「繼續」後，這則訊息日後就不會再出現。以下是這個初次畫面的樣子：



佈建並連至 Cloud Shell 預計只需要幾分鐘。



這部虛擬機器搭載您需要的所有開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。本程式碼研究室幾乎所有工作都可在瀏覽器或 Chromebook 上完成。

連線至 Cloud Shell 後，您應會發現自己通過驗證，且專案已設為您的專案 ID。

2. 在 Cloud Shell 中執行下列指令，確認您已通過驗證：

```
gcloud auth list
```

指令輸出

Credentialed Accounts

ACTIVE ACCOUNT

* <my_account>@<my_domain.com>

To set the active account, run:

```
$ gcloud config set account `ACCOUNT`
```

注意： `gcloud` 指令列工具是 Google Cloud 中功能強大的整合式指令列工具，並已預先安裝於 Cloud Shell。你會發現它支援 Tab 鍵自動完成功能。詳情請參閱 [gcloud 指令列工具總覽](#)。

3. 在 Cloud Shell 中執行下列指令，確認 `gcloud` 指令知道您的專案：

```
gcloud config list project
```

指令輸出

```
[core]
project = <PROJECT_ID>
```

如未設定，請輸入下列指令手動設定專案：

```
gcloud config set project <PROJECT_ID>
```

指令輸出

```
Updated property [core/project].
```

步驟 3 · 約 2 分鐘

3. 初始化 Spring Boot 應用程式

1. 使用 Spring Initializr 產生新的 Spring Boot 應用程式。

```
$ curl https://start.spring.io/starter.tgz \
  -d language=kotlin \
  -d dependencies=web \
  -d baseDir=kotlin-jib-cloud-run | tar -xzvf -
```

請注意，Initializr 會自動將 `spring-boot-starter-web` 新增至範本應用程式 `pom.xml` 中的依附元件。

2. 變更為範本應用程式的目錄。

```
$ cd kotlin-jib-cloud-run
```

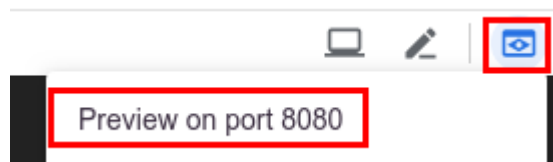
3. 使用 Maven 建構並執行應用程式。

```
$ ./mvnw -DskipTests spring-boot:run
```

4. 啟動後，應用程式會開始監聽通訊埠 8080。按一下 Cloud Shell 工具列中的「網頁預覽」



，然後選取「透過以下通訊埠預覽：8080」，即可存取應用程式。



5. 由於應用程式尚未執行任何實用動作，因此您應該會收到 404 回應。使用 `Control+C` 停止應用程式。

步驟 4 · 約 3 分鐘

4. 新增網頁控制器

1. 在示範套件中建立下列 `Controller` 類別：

```
$ vi src/main/kotlin/com/example/demo/Controller.kt
or
$ nano src/main/kotlin/com/example/demo/Controller.kt
```

`src/main/kotlin/com/example/demo/Controller.kt`

```
package com.example.demo

import org.springframework.web.bind.annotation.GetMapping
import org.springframework.web.bind.annotation.RestController

@RestController
class Controller {

    @GetMapping("/")
    fun saySomething(): String {
        return "Kotlin app on Cloud Run, containerized by Jib!"
    }
}
```

2. 重新建構並執行應用程式。

```
$ ./mvnw spring-boot:run
```

3. 使用「網頁預覽」



再次檢查應用程式。這次您應該會看到「`Kotlin app on Cloud Run, containerized by Jib!`」訊息。使用 `Control+C` 停止應用程式。

步驟 5 · 約 3 分鐘

5. 將應用程式容器化並發布至 Container Registry

使用 [Jib](#)，您可以在不使用 Docker 的情況下，以最佳化方式將應用程式容器化，並發布至任何容器登錄檔。

1. 請先啟用 Container Registry API，再繼續操作。每項專案只需要執行這項操作一次，即可存取 API。

```
$ gcloud services enable containerregistry.googleapis.com
```

2. 執行 Jib 建構 Docker 映像檔，並發布至 Container Registry。

```
$ ./mvnw com.google.cloud.tools:jib-maven-plugin:3.1.1:build \
-Dimage=gcr.io/$GOOGLE_CLOUD_PROJECT/kotlin-jib-cloud-run
```

注意：在本程式碼研究室中，您並未在 `pom.xml` 中設定 Jib Maven 外掛程式，但如要進階使用，可以將其新增至 `pom.xml`，並設定更多選項。

最後，您會看到以下訊息，表示應用程式已容器化並推送至 Container Registry。

```
[INFO] Built and pushed image as gcr.io/PROJECT_ID/kotlin-jib-cloud-run
...
[INFO] BUILD SUCCESS
```

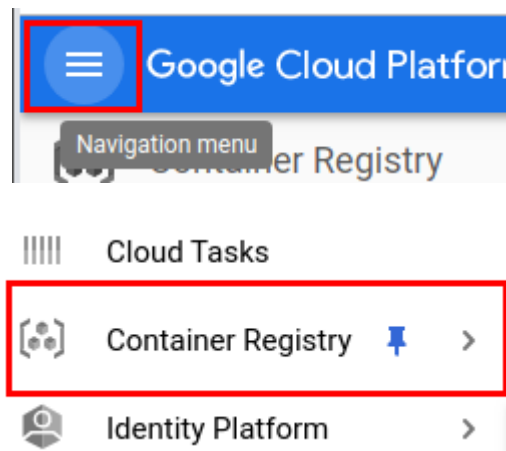
如果看到錯誤訊息，請仔細檢查 `$GOOGLE_CLOUD_PROJECT` 是否已正確設為 Google Cloud 專案 ID (`PROJECT_ID`)。

注意：在本程式碼研究室中，Jib 會直接將映像檔發布至 Container Registry，但 Jib 也有選項可推送至本機 Docker Daemon 或建立映像檔 tarball。

3. 繼續操作前，請先確認圖片是否已成功發布。返回 [Cloud 控制台](#)，按一下「導覽選單」



，然後選取「Container Registry」。



你會看到圖片已成功發布。

Filter	
Name ^	Hostname
 <u>kotlin-jib-cloud-run</u> 	gcr.io

步驟 6 · 約 3 分鐘

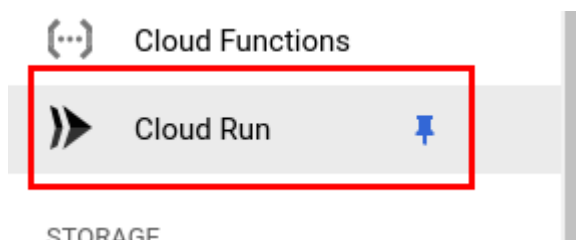
6. 在 Cloud Run 執行容器化應用程式

Cloud Run 將無伺服器技術導入容器，可自動調整無狀態容器的資源配置。

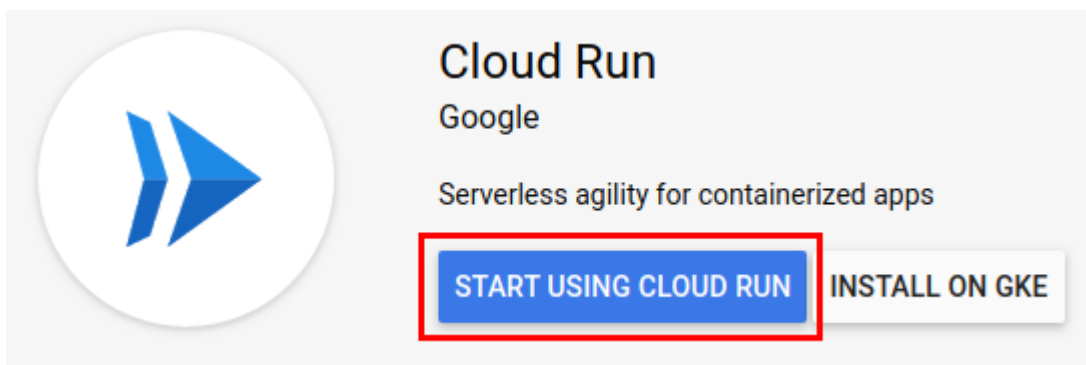
1. 再次按一下「導覽選單」



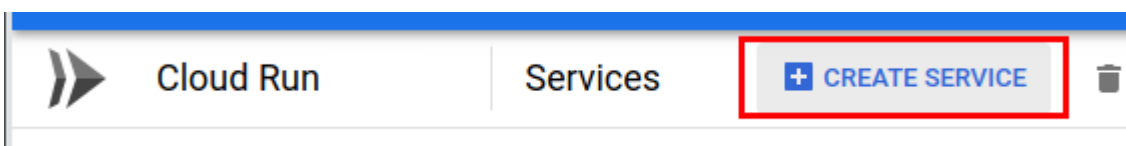
，然後選取「Cloud Run」。



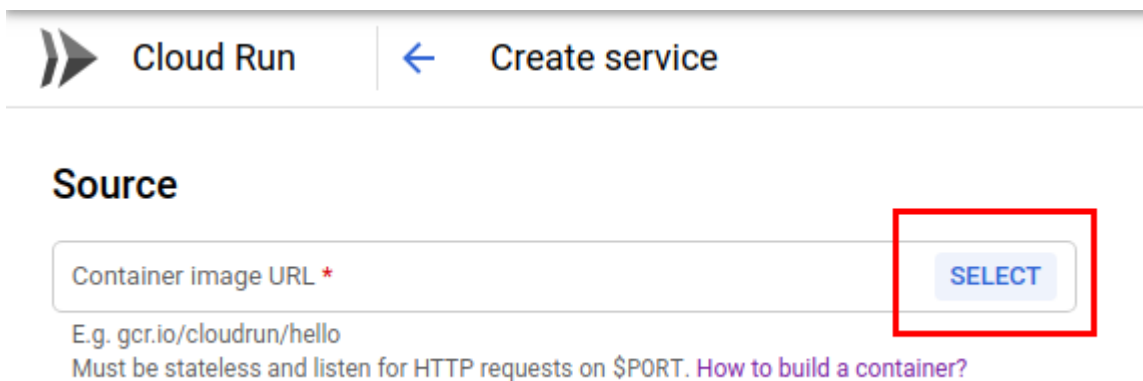
如果您是第一次存取 Cloud Run，系統會顯示下列對話方塊，要求您完成一次性設定。如果看到「開始使用 Cloud Run」，請按一下。



2. 在 Cloud Run 頁面中，按一下「建立服務」。



3. 在下一個畫面中，按一下「來源」下方的「選取」。來源是您要在 Cloud Run 上執行的映像檔。



4. 對話方塊會顯示您先前建構的映像檔。選取圖片，然後按一下「繼續」。

Select Google Container Registry Image

Project: [redacted]

▼ kotlin-jib-cloud-run

24a8b89711 latest

3 minutes ago

CANCEL

CONTINUE

5. 只要按幾下滑鼠，就能立即部署應用程式。選取「Deployment platform」(部署平台) 下方的「Cloud Run (fully managed)」(Cloud Run (全代管))，即可在 Google Cloud 上全代管服務。選擇適合您所在位置的區域，選取「Allow unauthenticated invocations」(允許未經驗證的叫用)，然後按一下「Create」(建立)。就是這麼簡單！

Authentication *

- ☒ Allow unauthenticated invocations
Check this if you are creating a public API or website.
- ☐ Require authentication
Manage authorized users with Cloud IAM.

✓ SHOW OPTIONAL REVISION SETTINGS

CREATE

CANCEL

映像檔完全部署後，Cloud Run 頁面會顯示存取應用程式的網址。快來看看吧！

Cloud Run

Service details

DEPLOY NEW REVISION

✓ kotlin-jib-cloud-run

Region: us-central1

URL: <https://kotlin-jib-cloud-run-yg64dfalmq-uc.a.run.app>

METRICS

REVISIONS

LOGS

DETAILS

YAML

Revisions

Filter revisions

	Name	Traffic	Deployed	Actions
✓	kotlin-jib-cloud-run-00001	100%	Just now	⋮

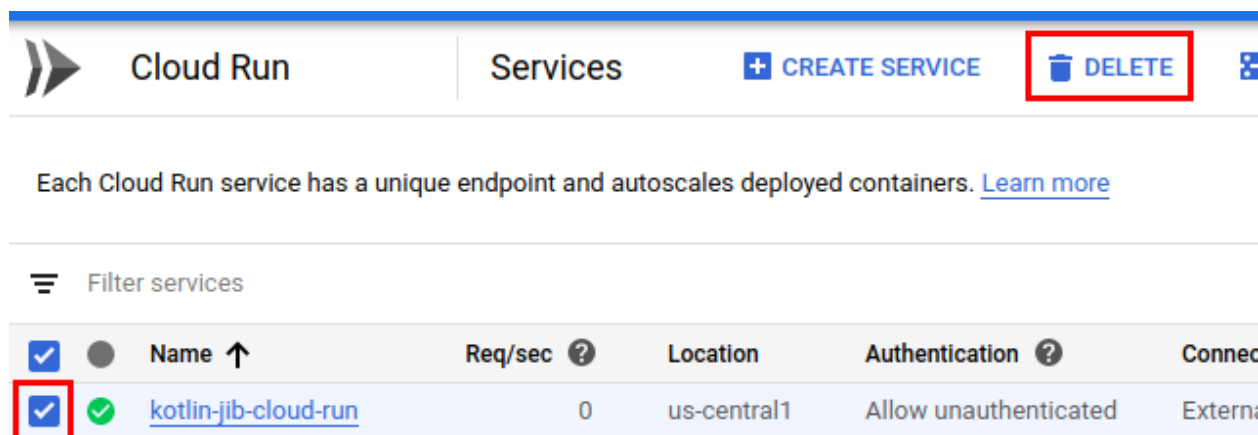
最後，您會看到應用程式傳送的預期訊息。

Kotlin app on Cloud Run, containerized by Jib!

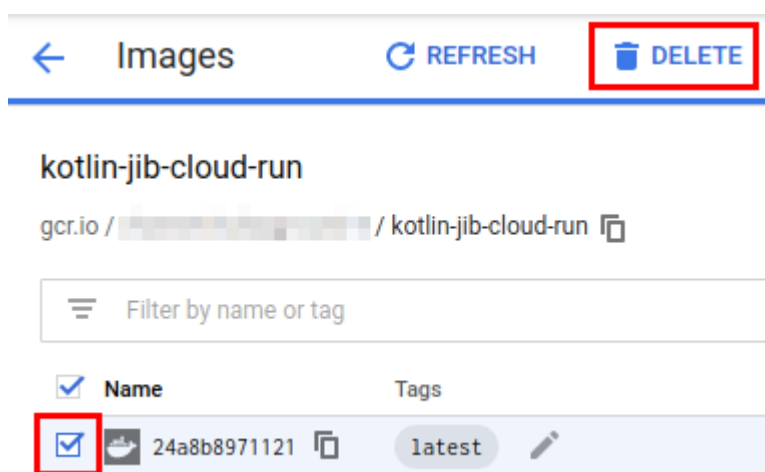
大功告成！日後如需部署新版應用程式，請點選頁面上的「Deploy New Revision」(部署新修訂版本)。

7. 清除所用資源

1. 如要清除環境，請刪除 Cloud Run 上部署的應用程式，以及 Container Registry 中發布的映像檔。前往 **Cloud Run**，選取應用程式，然後按一下「刪除」。



2. 同樣地，前往 Container Registry 頁面並刪除映像檔。



8. 恭喜

恭喜！您已成功將 Spring Boot Kotlin 應用程式容器化，並部署至 Cloud Run！

您使用 Jib 建構了最佳化容器映像檔，不必安裝 Docker 或編寫 Dockerfile，並將映像檔發布至 Container Registry。Jib 會將映像檔建構作業最佳化，因此即使沒有深入的 Docker 知識，也能快速有效率地將 Java 應用程式容器化。接著按幾下滑鼠，將應用程式部署至 Cloud Run，即可立即開始提供服務。

瞭解詳情

- [將 Java 應用程式部署至 Google Kubernetes Engine 上的 Kubernetes](#)
 - [Cloud Run 說明文件](#)
 - [Cloud Run 總覽](#)
 - [Jib 簡介—建構更優質的 Java Docker 映像檔](#)
 - [使用 Jib \(適用於 Java 應用程式的 Google 映像檔建構工具\) 加快容器建構速度](#)
 - [Jib - 將 Java 應用程式容器化](#)
 - [Jib Gitter 頻道](#)
 - [Jib 使用者郵寄清單](#)
-